

FRC DOCS • ROBOT SIMULATION

Robot Simulation 机器人仿真

中英对照合订本

按 WPILib 文档索引顺序 • 10 篇

Contents / 目录

01	Introduction to Robot Simulation / 机器人仿真简介
02	Simulation Specific User Interface Elements / 仿真专用用户界面元素
03	Physics Simulation with WPILib / 使用 WPILib 进行物理仿真
04	Device Simulation / 设备仿真
05	Drivetrain Simulation Overview / 底盘仿真概览
06	Step 1: Creating Simulated Instances of Hardware / 第 1 步：创建硬件的仿真实例
07	Step 2: Creating a Drivetrain Model / 第 2 步：创建底盘模型
08	Step 3: Updating the Drivetrain Model / 第 3 步：更新底盘模型
09	Step 4: Updating Odometry and Visualizing Robot Position / 第 4 步：更新里程计并可视化机器人位置
10	Unit Testing / 单元测试

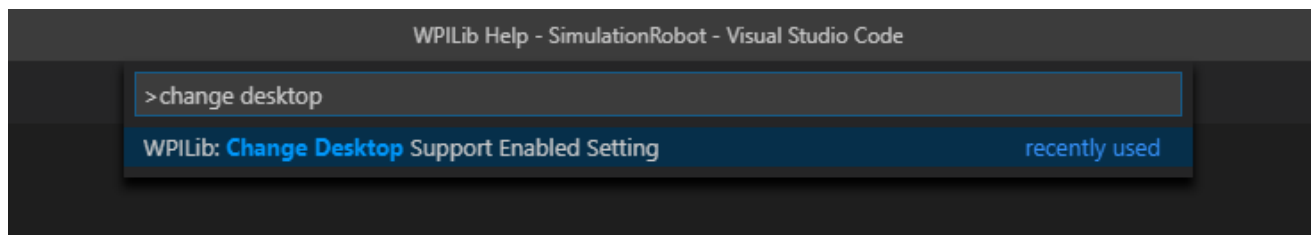
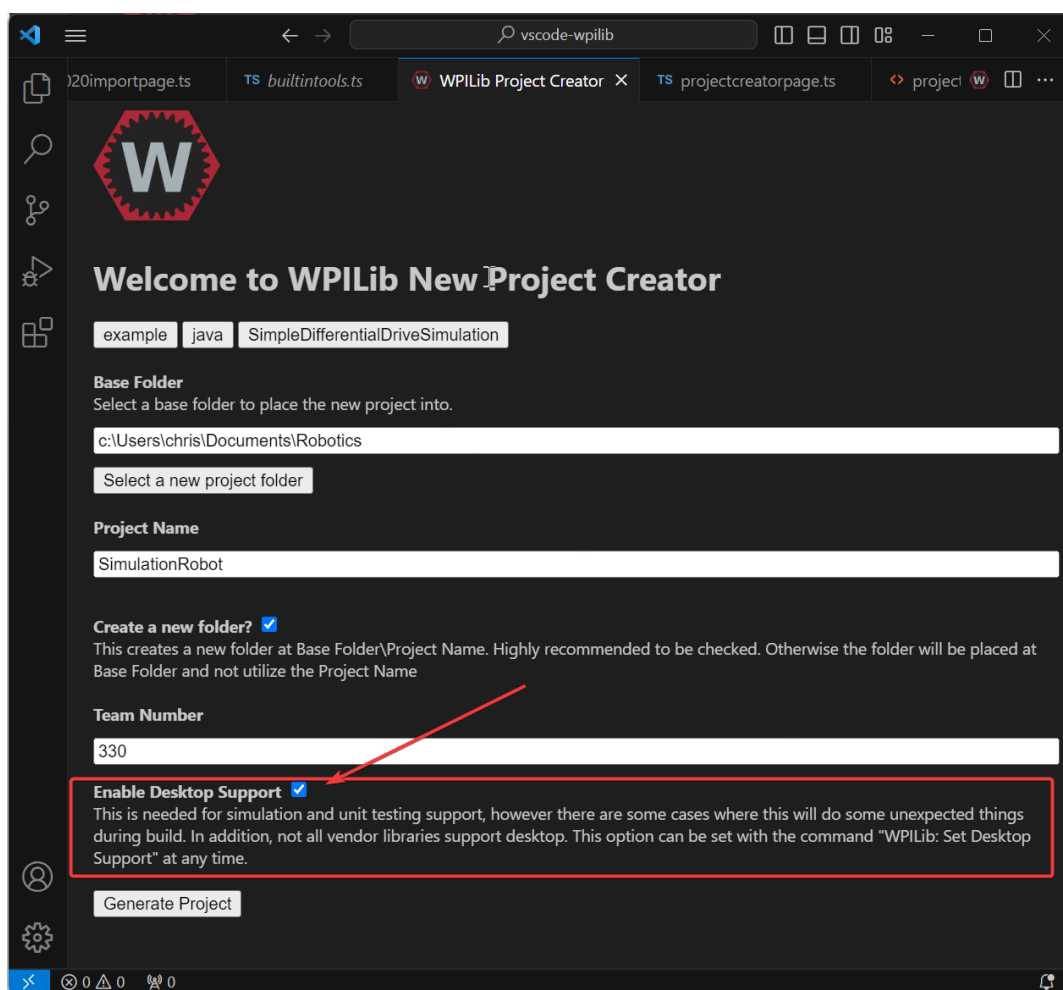
Introduction to Robot Simulation / 机器人仿真简介

Often a team may want to test their code without having an actual robot available. WPILib provides teams with the ability to simulate various robot features using simple gradle commands.

队伍常常希望在没有实体机器人的情况下测试代码。WPILib 使队伍能够通过简单的 Gradle 命令仿真机器人的多种功能。

Use of the Desktop Simulator requires Desktop Support to be enabled. This can be done by checking the “Enable Desktop Support Checkbox” when creating your robot project or by running “WPILib: Change Desktop Support Enabled Setting” from the Visual Studio Code command palette.

使用桌面仿真器需要启用 Desktop Support。可以在创建机器人项目时勾选 “Enable Desktop Support Checkbox”，也可以从 Visual Studio Code 的命令面板运行 “WPILib: Change Desktop Support Enabled Setting”。



Desktop support can also be enabled by manually editing your `build.gradle` file located at the root of your robot project. Simply change `includeDesktopSupport = false` to `includeDesktopSupport = true`

也可以手动编辑机器人项目根目录中的 `build.gradle` 文件，以启用 Desktop Support。只需将 `includeDesktopSupport = false` 改为 `includeDesktopSupport = true`。

Important / 重要

It is important to note that enabling desktop/simulation support can have unintended consequences. Not all vendors will support this option, and code that uses their libraries may even crash when attempting to run simulation!

请注意，启用桌面/仿真支持可能产生意料之外的后果。并非所有供应商都支持此选项，使用其库的代码在尝试运行仿真时甚至可能崩溃！

If at any point in time you want to disable Desktop Support, simply re-run the “WPILib: Change Desktop Support Enabled Setting” from the command palette or change `includeDesktopSupport` to `false` in `build.gradle`.

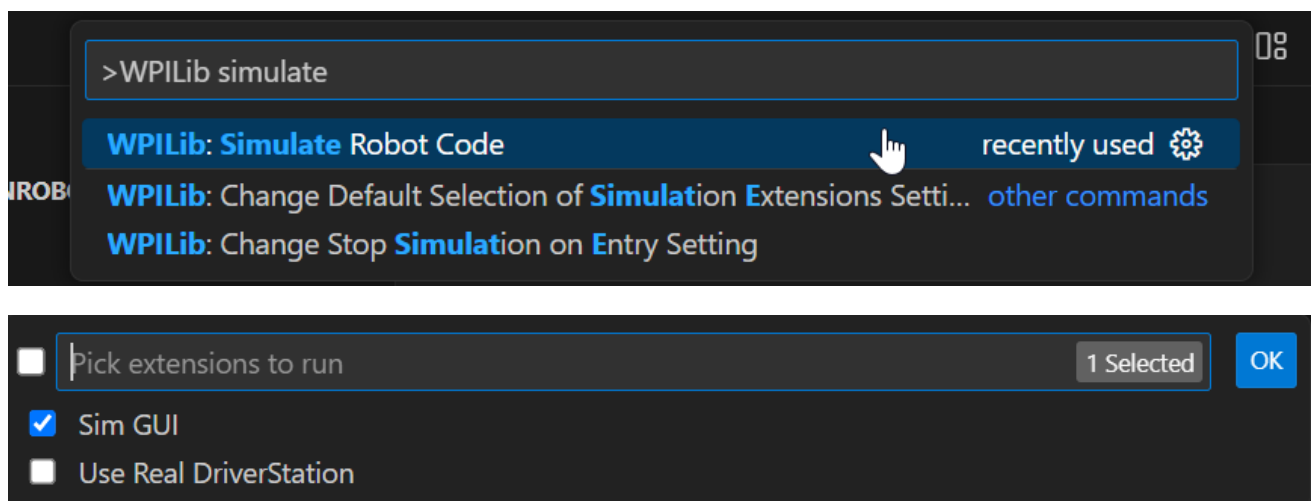
如果之后想禁用 Desktop Support，只需再次从命令面板运行 “WPILib: Change Desktop Support Enabled Setting”，或在 `build.gradle` 中将 `includeDesktopSupport` 改为 `false`。

Running Robot Simulation

运行机器人仿真

Basic robot simulation can be run using VS Code. This can be done by using VS Code’s command palette WPILib: Simulate Robot Code as shown below.

可以使用 VS Code 运行基础的机器人仿真。如图所示，从 VS Code 命令面板选择 WPILib: Simulate Robot Code 即可。



The Sim GUI option will be selected by default. This will launch the [Simulation GUI](#). You can also launch simulation without a GUI by unchecking Sim GUI in which case your console output in Visual Studio Code should look like the below.

默认会选中 Sim GUI 选项。它将启动[仿真 GUI](#)。也可以取消勾选 Sim GUI，以无 GUI 模式启动仿真；此时 Visual Studio Code 中的控制台输出应如下所示。

```
***** Robot program starting *****  
Default disabledInit() method... Override me!  
Default disabledPeriodic() method... Override me!  
Default robotPeriodic() method... Override me!
```

If you would like to prevent the pop up and only use build.gradle to configure your simulation settings you can add the following in your vscode settings.json file.

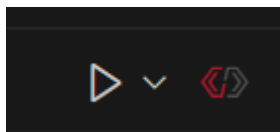
如果希望不再出现弹出窗口，而只通过 build.gradle 配置仿真设置，可以在 VS Code 的 settings.json 文件中添加以下内容。

```
"wpilib.skipSelectSimulateExtension": true
```

Warning / 警告

You may see a run button next to the WPILib button. This button does not set up simulation appropriately and should not be used. Instead, the menu item shown above WPILib: Simulate Robot Code should be used.

你可能会在 WPILib 按钮旁看到一个运行按钮。该按钮不会正确设置仿真，不应使用。应使用上图所示的菜单项 WPILib: Simulate Robot Code。



Note / 说明

Simulation can also be run outside of VS Code using `./gradlew simulateJava` for Java.

在 VS Code 之外，也可以使用 Java 的 `./gradlew simulateJava` 命令运行仿真。

Note / 说明

Some vendors support attaching hardware to your PC and using the hardware in desktop simulation (e.g. CANivore). See [vendor documentation](#) for more information about the command WPILib: Hardware Sim Robot Code.

部分供应商支持将硬件连接到 PC，并在桌面仿真中使用该硬件（例如 CANivore）。有关 WPILib: Hardware Sim Robot Code 命令的更多信息，请参阅[供应商文档](#)。

Running Robot Dashboards / 运行机器人仪表板

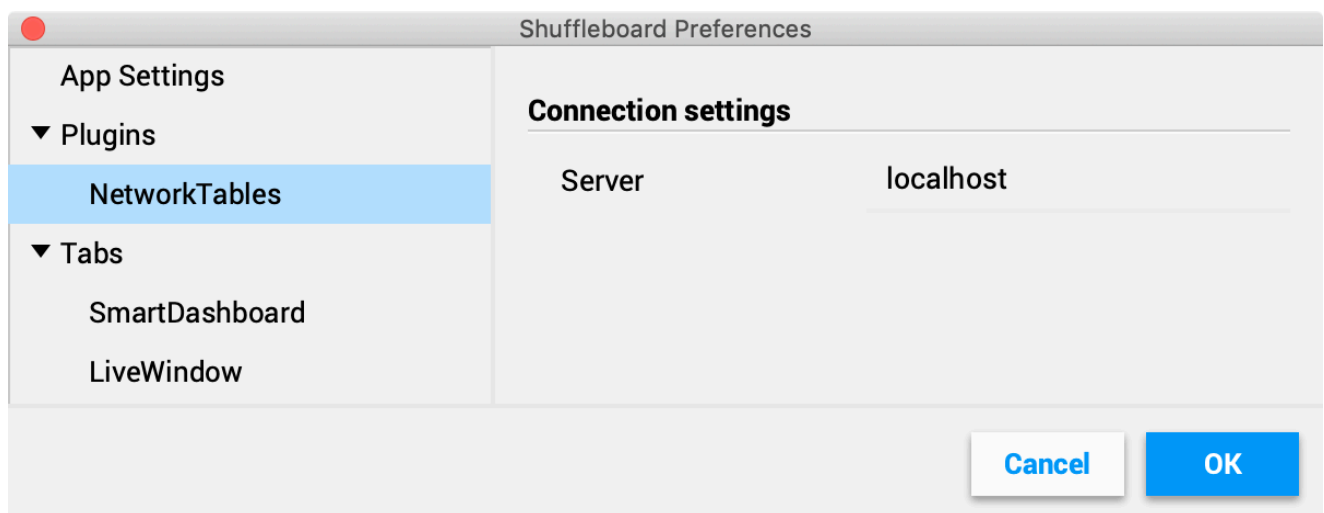
Shuffleboard, SmartDashboard, Glass, and AdvantageScope can be used with WPILib simulation when they are configured to connect to the local computer (i.e. localhost).

将 Shuffleboard、SmartDashboard、Glass 和 AdvantageScope 配置为连接本地计算机（即 localhost）后，即可将它们与 WPILib 仿真配合使用。

Shuffleboard / Shuffleboard

Shuffleboard is automatically configured to look for a NetworkTables instance from the robotRIO but **not from other sources**. To connect to a simulation, open Shuffleboard preferences from the File menu and select NetworkTables under Plugins on the left navigation bar. In the Server field, type in the IP address or hostname of the NetworkTables host. For a standard simulation configuration, use localhost.

Shuffleboard 会自动查找来自 roboRIO 的 NetworkTables 实例，但**不会自动查找其他来源的实例**。要连接到仿真，请从 File 菜单打开 Shuffleboard preferences，在左侧导航栏的 Plugins 下选择 NetworkTables。在 Server 字段中输入 NetworkTables 主机的 IP 地址或主机名。对于标准仿真配置，使用 localhost。



SmartDashboard / SmartDashboard

SmartDashboard is automatically configured to look for a NetworkTables instance from the roboRIO, but **not from other sources**. To connect to a simulation, open SmartDashboard preferences under the File menu and in the Team Number field, enter the IP address or hostname of the NetworkTables host. For a standard simulation configuration, use localhost.

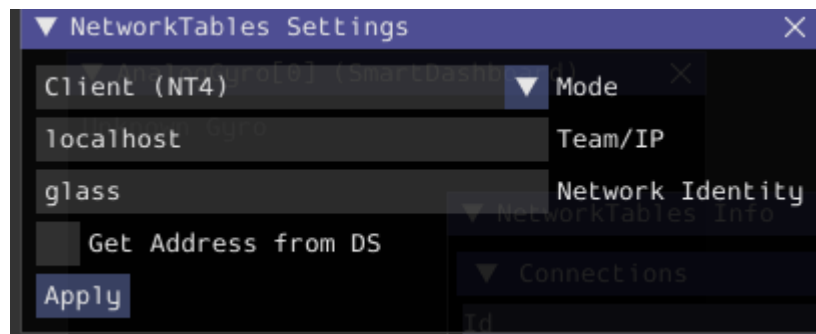
SmartDashboard 会自动查找来自 roboRIO 的 NetworkTables 实例，但**不会自动查找其他来源的实例**。要连接到仿真，请从 File 菜单打开 SmartDashboard preferences，并在 Team Number 字段中输入 NetworkTables 主机的 IP 地址或主机名。对于标准仿真配置，使用 localhost。

Edit Preferences	
Property	Value
Team Number	localhost
Hide Menu	☐
Automatically Show Widgets	☒
Grid Cell Width(s)	16
Grid Cell Height(s)	16
Window X Position	723
Window Y Position	432
Window Width	317
Window Height	200
Save File	/example
Log to CSV	☐
CSV File	/example

Glass / Glass

Glass is automatically configured to look for a NetworkTables instance from the roboRIO, but **not from other sources**. To connect to a simulation, open NetworkTables Settings under the NetworkTables menu and in the Team/IP field, enter the IP address or hostname of the NetworkTables host. For a standard simulation configuration, use localhost.

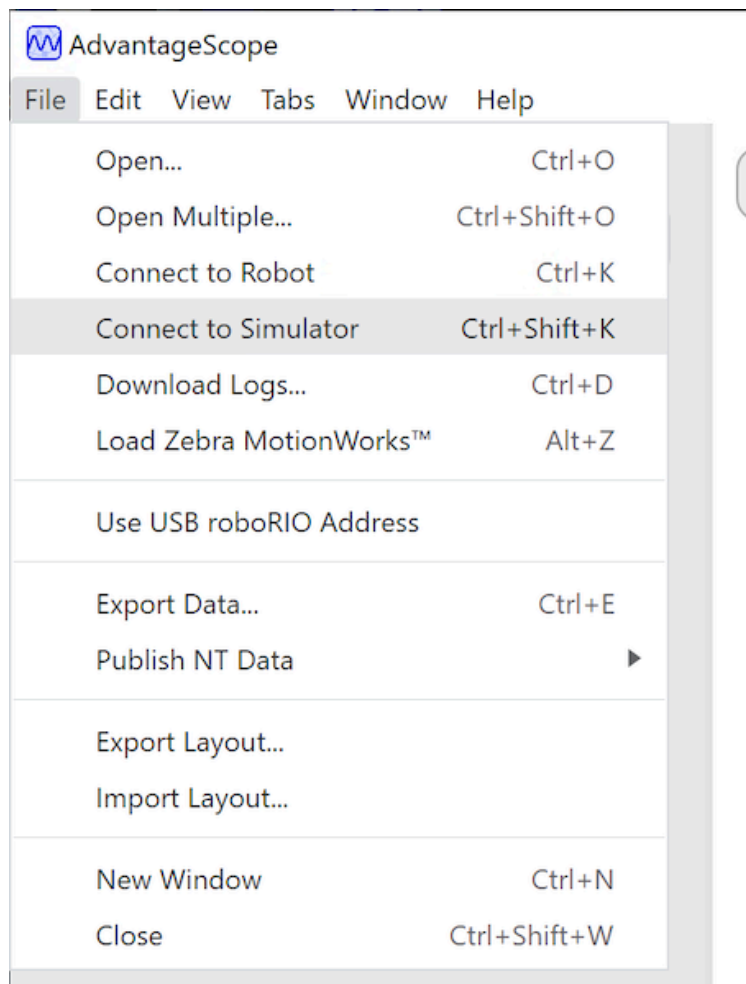
Glass 会自动查找来自 roboRIO 的 NetworkTables 实例，但**不会自动查找其他来源的实例**。要连接到仿真，请从 NetworkTables 菜单打开 NetworkTables Settings，并在 Team/IP 字段中输入 NetworkTables 主机的 IP 地址或主机名。对于标准仿真配置，使用 localhost。



AdvantageScope / AdvantageScope

No configuration is required to connect to a NetworkTables instance running on the local computer. To connect to a simulation, click Connect to Simulator under the File menu or press **Ctrl+Shift+K**.

连接到本地计算机上运行的 NetworkTables 实例无需任何配置。要连接到仿真，请在 File 菜单中单击 Connect to Simulator，或按 **Ctrl+Shift+K**。



Simulation Specific User Interface Elements / 仿真专用用户界面元素

WPILib has extended robot simulation to introduce a graphical user interface (GUI) component. This allows teams to easily visualize their robot's inputs and outputs.

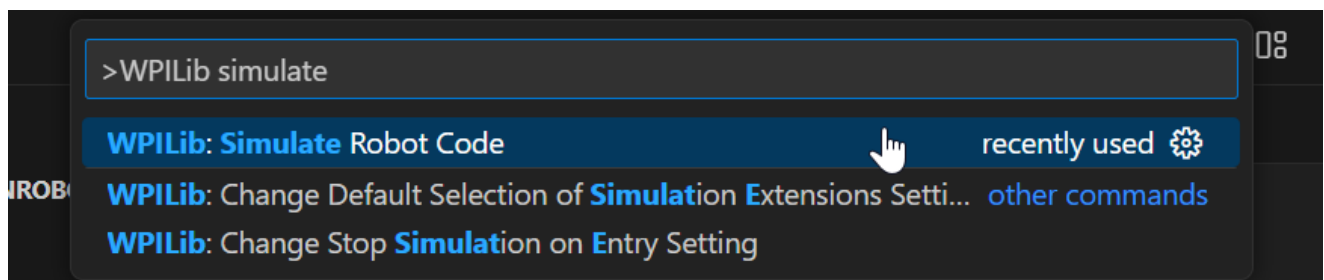
WPILib 扩展了机器人仿真功能，加入了图形用户界面（GUI）组件，使队伍能够方便地查看机器人的输入和输出。

Note / 说明

The Simulation GUI is very similar in many ways to [Glass](#). Some of the following pages will link to Glass sections that describe elements common to both GUIs.

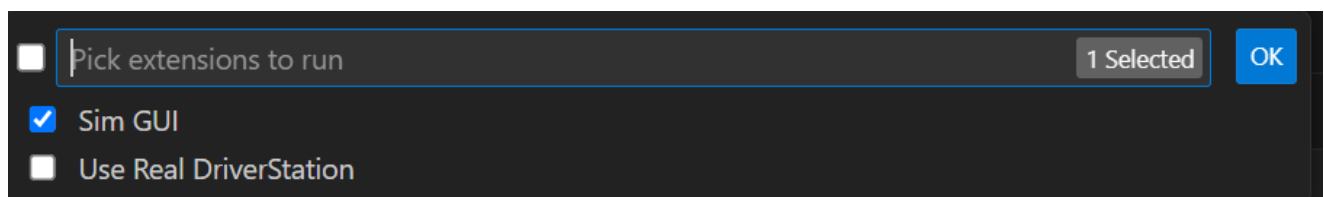
仿真 GUI 在许多方面与 [Glass](#) 非常相似。后续部分页面会链接到 Glass 章节，以介绍两个 GUI 共有的元素。

Running the GUI / 运行 GUI



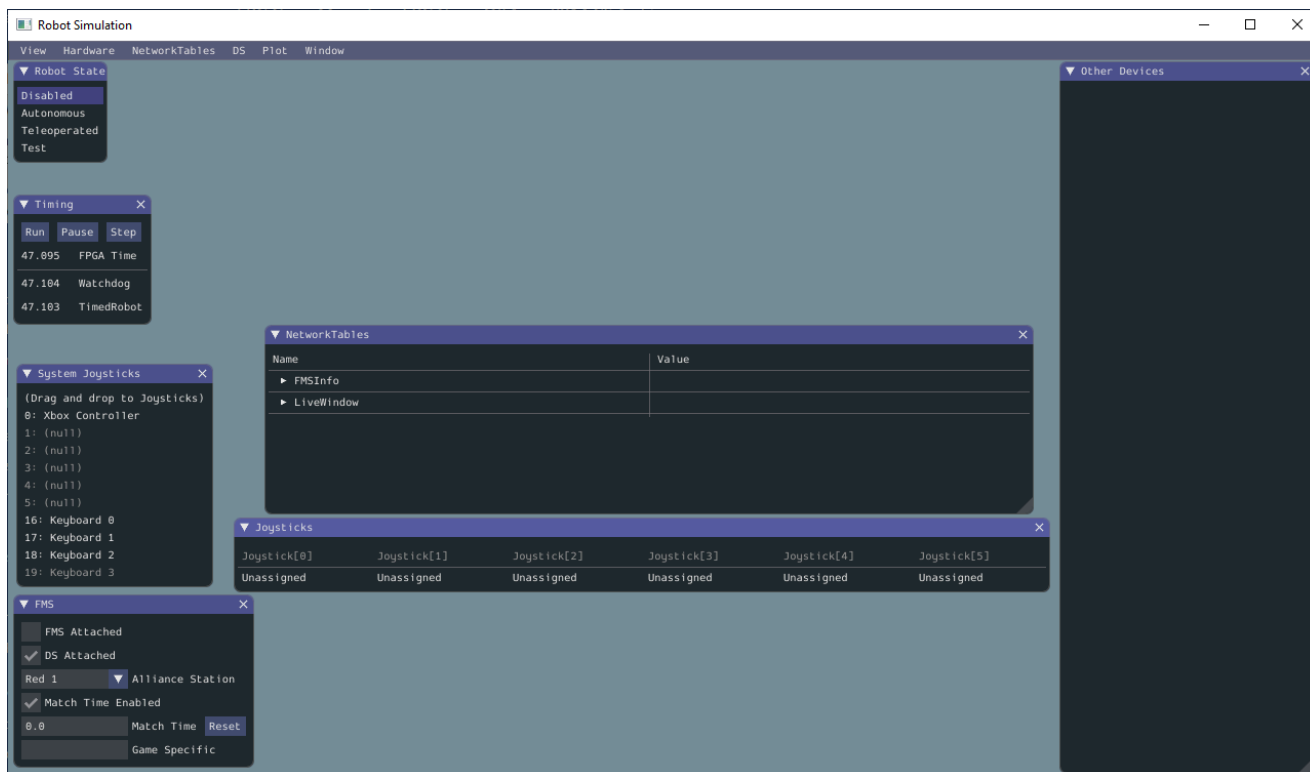
You can simply launch the GUI via the Simulate Robot Code command palette option.

只需在命令面板中选择 Simulate Robot Code，即可启动 GUI。



And the Sim GUI option should popup in a new dialog and will be selected by default. Press Ok. This will now launch the Simulation GUI.

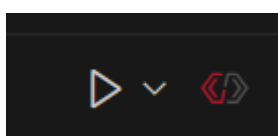
Sim GUI 选项会在新对话框中弹出，并且默认选中。按 Ok 即可启动仿真 GUI。



Warning / 警告

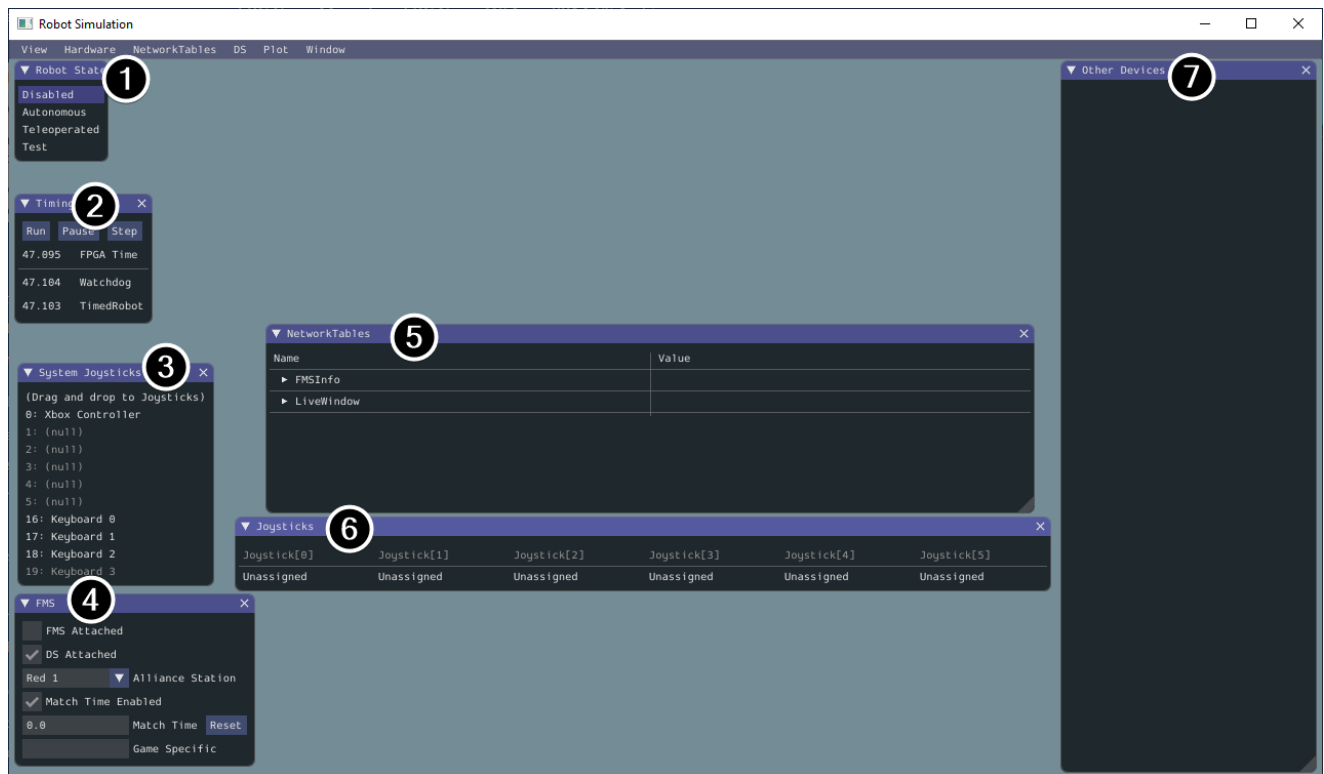
You may see a run button next to the WPILib button. This button does not set up simulation appropriately and should not be used. Instead, the menu item shown above WPILib: Simulate Robot Code should be used.

你可能会在 WPILib 按钮旁看到一个运行按钮。该按钮不会正确设置仿真，不应使用。应使用上图所示的菜单项 WPILib: Simulate Robot Code。



Using the GUI / 使用 GUI

Learning the Layout / 了解界面布局



The following items are shown on the simulation GUI by default:

仿真 GUI 默认显示以下项目：

1. **Robot State** - This is the robot's current state or "mode". You can click on the labels to change mode as you would on the normal Driver Station.
1. **Robot State** - 机器人的当前状态或“模式”。可以像在普通 Driver Station 中一样，单击标签切换模式。
2. **Timing** - Shows the values of the Robot's timers and allows the timing to be manipulated.
2. **Timing** - 显示机器人计时器的值，并允许调整计时。
3. **System Joysticks** - This is a list of joysticks connected to your system currently.
3. **System Joysticks** - 当前连接到计算机的摇杆列表。
4. **FMS** - This is used for simulating many of the common [FMS](#) systems.
4. **FMS** - 用于仿真许多常见的 [FMS](#) 系统。
5. **NetworkTables** - This shows the data that has been published to NetworkTables.
5. **NetworkTables** - 显示已发布到 NetworkTables 的数据。
6. **Joysticks** - This is joysticks that the robot code can directly pull from.
6. **Joysticks** - 机器人代码能够直接读取的摇杆。
7. **Other Devices** - This includes devices that do not fall into any of the other categories, such as the ADXRS450 gyro that is included in the Kit of Parts or third party devices that support simulation.
7. **Other Devices** - 包括不属于其他类别的设备，例如 Kit of Parts 中的 ADXRS450 陀螺仪，或支持仿真的第三方设备。

The following items can be added from the Hardware menu, but are not shown by default.

可以从 Hardware 菜单添加以下项目，但它们默认不显示。

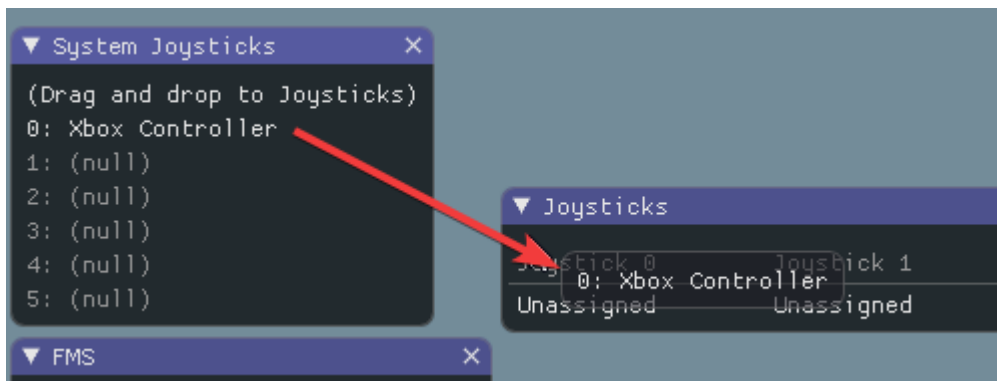
1. **Addressable LEDs** - This shows LEDs controlled by the AddressableLED Class.

1. **Addressable LEDs** - 显示由 AddressableLED 类控制的 LED。
2. **Analog Inputs** - This includes any devices that would normally use the **ANALOG IN** connector on the roboRIO, such as any Analog based gyros.
2. **Analog Inputs** - 包括通常使用 roboRIO 上 **ANALOG IN** 接口的设备，例如模拟陀螺仪。
3. **DIO** - (Digital Input Output) This includes any devices that use the **DIO** connector on the roboRIO.
3. **DIO** - (数字输入输出) 包括使用 roboRIO 上 **DIO** 接口的设备。
4. **Encoders** - This will show any instantiated devices that extend or use the Encoder class.
4. **Encoders** - 显示实例化的、继承或使用 Encoder 类的设备。
5. **PDPs** - This shows the Power Distribution Panel object.
5. **PDPs** - 显示 Power Distribution Panel 对象。
6. **PWM Outputs** - This is a list of instantiated [PWM](#) devices. This will appear as many devices as you instantiate in robot code, as well as their outputs.
6. **PWM Outputs** - 实例化的 [PWM](#) 设备列表。机器人代码中实例化多少设备及其输出，这里就会显示多少。
7. **Relays** - This includes any relay devices. This includes VEX Spike relays.
7. **Relays** - 包括所有继电器设备，其中包括 VEX Spike 继电器。
8. **Solenoids** - This is a list of “connected” solenoids. When you create a solenoid object and push outputs, these are shown here.
8. **Solenoids** - “已连接”的电磁阀列表。创建电磁阀对象并输出值后，它们会显示在这里。

Adding a System Joystick to Joysticks / 将系统摇杆添加到 Joysticks

To add a joystick from the list of system joysticks, simply click and drag a shown joystick under the “System Joysticks” menu to the “Joysticks” menu”.

要从系统摇杆列表添加摇杆，只需把“System Joysticks”菜单下显示的摇杆拖到“Joysticks”菜单中。



Note / 说明

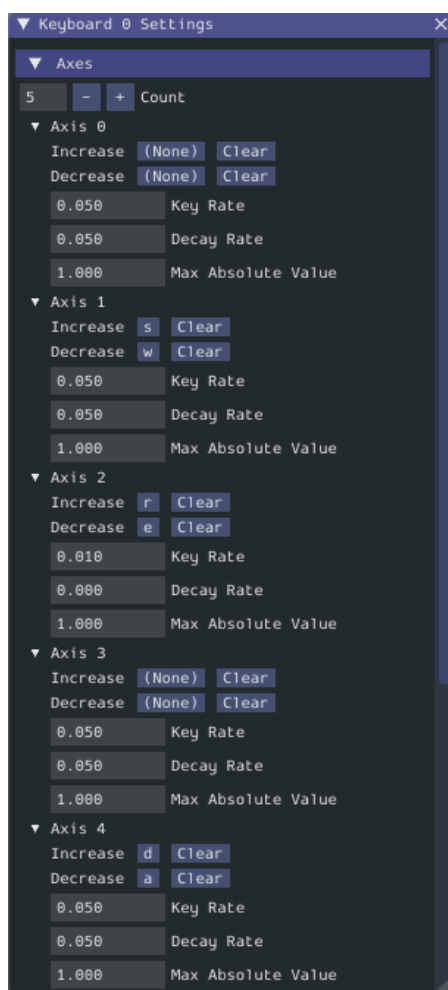
The FRC® Driver Station does special mapping to gamepads connected and the WPILib simulator does not “map” these by default. You can turn on this behavior by pressing the “Map gamepad” toggle underneath the “Joysticks” menu.

FRC® Driver Station 会对连接的游戏手柄进行特殊映射，而 WPILib 仿真器默认不会执行此映射。可以按“Joysticks”菜单下的“Map gamepad”开关启用这一行为。

Using the Keyboard as a Joystick / 将键盘用作摇杆

You add a keyboard to the list of system joysticks by clicking and dragging one of the keyboard items (e.g. Keyboard 0) just like a joystick above. To edit the settings of the keyboard go to the DS item in the menu bar then choose Keyboard 0 Settings. This allows you to control which keyboard buttons control which axis. This is a common example of how to make the keyboard similar to a split sticks arcade drive on an Xbox controller (uses axis 1 & 4):

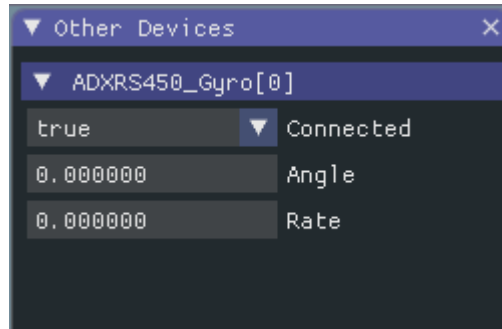
与添加上述摇杆一样，可以拖动一个键盘项目（例如 Keyboard 0），将键盘添加到摇杆列表。要编辑键盘设置，请在菜单栏中打开 DS 项目，然后选择 Keyboard 0 Settings。这可用于配置各键盘按键控制哪个轴。以下是让键盘模拟 Xbox 手柄分离式摇杆街机驱动的一种常见设置（使用轴 1 和轴 4）：



Modifying ADXRS450 Inputs / 修改 ADXRS450 输入

Using the ADXRS450 object is a fantastic way to test gyro based outputs. This will show up in the “Other Devices” menu. A drop down menu is then exposed that shows various options such as “Connected”, “Angle”, and “Rate”. All of these values are values that you can change, and that your robot code can use on-the-fly.

使用 ADXRS450 对象是测试基于陀螺仪的输出的一种有效方式。它会出现在“Other Devices”菜单中。展开下拉菜单后，可看到“Connected”“Angle”和“Rate”等选项。所有这些值都可以修改，机器人代码也可以实时使用它们。



Determining Simulation from Robot Code / 在机器人代码中判断是否处于仿真

In cases where vendor libraries do not compile when running the robot simulation, you can wrap their content with `RobotBase.isReal()` which returns a boolean.

如果供应商库在运行机器人仿真时无法编译，可以用返回 boolean 的 `RobotBase.isReal()` 将相关内容包裹起来。

```
TalonSRX motorLeft;
TalonSRX motorRight;
public Robot() {
    if (RobotBase.isReal()) {
        motorLeft = new TalonSRX(0);
        motorRight = new TalonSRX(1);
    }
}
```

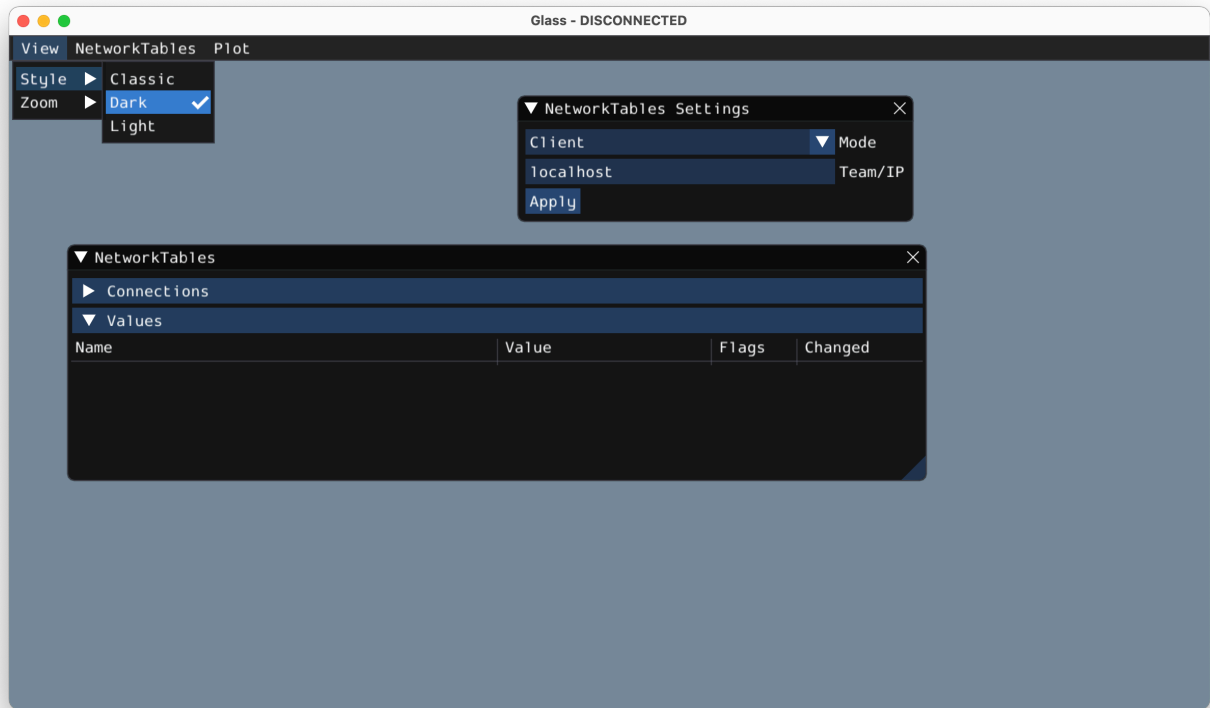
Changing View Settings / 更改视图设置

The View menu item contains Zoom and Style settings that can be customized. The Zoom option dictates the size of the text in the application whereas the Style option allows you to select between the Classic, Light, and Dark modes.

View 菜单包含可自定义的 Zoom 和 Style 设置。Zoom 选项控制应用中的文字大小；Style 选项可在 Classic、Light 和 Dark 模式之间切换。

An example of the Dark style setting is below:

下图是 Dark 样式的示例：



Clearing Application Data / 清除应用数据

Application data for the Simulation GUI, including widget sizes and positions as well as other custom information for widgets is stored in a `imgui.ini` file. This file is stored in the root of the project directory that the simulation is run from.

仿真 GUI 的应用数据（包括小组件的尺寸、位置及其他自定义信息）存储在 `imgui.ini` 文件中。该文件位于运行仿真的项目根目录。

The `imgui.ini` configuration file can simply be deleted to restore the Simulation GUI to a “clean slate” .

直接删除 `imgui.ini` 配置文件，即可将仿真 GUI 恢复到“初始状态”。

Physics Simulation with WPILib / 使用 WPILib 进行物理仿真

Because [state-space notation](#) allows us to compactly represent the [dynamics](#) of [systems](#), we can leverage it to provide a backend for simulating physical systems on robots. The goal of these simulators is to simulate the motion of robot mechanisms without modifying existing non-simulation user code. The basic flow of such simulators is as follows:

由于[状态空间表示法](#)能够简洁地表示[系统的动力学](#)，因此可以将其用作机器人物理系统仿真的后端。这些仿真器的目标是在不修改现有非仿真用户代码的情况下，仿真机器人机构的运动。此类仿真器的基本流程如下：

- In normal user code:
- 在普通用户代码中：
 - PID or similar control algorithms generate voltage commands from encoder (or other sensor) readings
 - PID 或类似控制算法根据编码器（或其他传感器）读数生成电压命令
 - Motor outputs are set
 - 设置电机输出
- In simulation periodic code:
- 在仿真周期代码中：
 - The simulation's [state](#) is updated using [inputs](#), usually voltages from motors set from a PID loop
 - 使用[输入](#)更新仿真的[状态](#)，输入通常是由 PID 循环设置的电机电压
 - Simulated encoder (or other sensor) readings are set for user code to use in the next timestep
 - 设置仿真编码器（或其他传感器）的读数，供用户代码在下一个时间步使用

WPILib's Simulation Classes / WPILib 的仿真类

The following physics simulation classes are available in WPILib:

WPILib 提供以下物理仿真类：

- LinearSystemSim, for modeling systems with linear dynamics
- LinearSystemSim，用于对具有线性动力学的系统建模
- FlywheelSim
- DifferentialDriveTrainSim
- ElevatorSim, which models gravity in the direction of elevator motion
- ElevatorSim，用于对升降机构运动方向上的重力影响建模
- SingleJointedArmSim, which models gravity proportional to the arm angle
- SingleJointedArmSim，用于对随机机械臂角度变化的重力影响建模
- BatterySim, which simply estimates battery voltage sag based on drawn currents
- BatterySim，根据消耗的电流简单估计电池电压下降

All simulation classes (with the exception of the differential drive simulator) inherit from the LinearSystemSim class. By default, the dynamics are the linear system dynamics ($\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k$). Subclasses override the `update(x, u, dt)` method to provide custom, nonlinear dynamics, such as modeling gravity.

除差速驱动仿真器外，所有仿真类都继承自 `LinearSystemSim` 类。默认情况下，其动力学遵循上面的线性系统方程。子类通过重写 `updateX(x, u, dt)` 方法，可提供自定义的非线性动力学，例如对重力建模。

Note / 说明

Swerve support for simulation is in the works, but we cannot provide an ETA. For updates on progress, please follow this [pull request](#).

swerve 仿真支持仍在开发中，但目前无法提供预计完成时间。有关进展，请关注此 [Pull Request](#)。

Usage in User Code / 在用户代码中使用

The following is available from the WPILib `elevatorsimulation` [example project](#).

以下内容来自 WPILib 的 `elevatorsimulation` [示例项目](#)。

In addition to standard objects such as motors and encoders, we instantiate our elevator simulator using known constants such as carriage mass and gearing reduction. We also instantiate an `EncoderSim`, which sets the distance and rate read by our `Encoder`.

除电机和编码器等标准对象外，我们还使用滑架质量和减速比等已知常量实例化升降机构仿真器。同时实例化一个 `EncoderSim`，由它设置 `Encoder` 读取的距离和速度。

In the following example, we simulate an elevator given the mass of the moving carriage (in kilograms), the radius of the drum driving the elevator (in meters), the gearing reduction between motor and drum as output over input (so usually greater than one), the minimum and maximum height of the elevator (in meters), the starting height of the elevator, and some random noise to add to our position estimate.

以下示例根据运动滑架的质量 (kg)、驱动升降机构的卷筒半径 (m)、电机与卷筒之间以输出对输入表示的减速比 (通常大于 1)、升降机构的最小和最大高度 (m)、初始高度，以及添加到位置估计中的随机噪声，来仿真升降机构。

Note / 说明

The elevator and arm simulators will prevent the simulated position from exceeding given minimum or maximum heights or angles. If you wish to simulate a mechanism with infinite rotation or motion, `LinearSystemSim` may be a better option.

升降机构和机械臂仿真器会防止仿真位置超出指定的最小或最大高度或角度。如果要仿真能够无限旋转或移动的机构，`LinearSystemSim` 可能更合适。

```
// Simulation classes help us simulate what's going on, including gravity.
private final ElevatorSim m_elevatorSim =
    new ElevatorSim(
        m_elevatorGearbox,
        Constants.kElevatorGearing,
        Constants.kCarriageMass,
        Constants.kElevatorDrumRadius,
        Constants.kMinElevatorHeightMeters,
```

```

        Constants.kMaxElevatorHeightMeters,
        true,
        0,
        0.01,
        0.0);

```

Next, `teleopPeriodic` uses a simple PID control loop to drive our elevator to a setpoint 30 inches off the ground.

接下来，`teleopPeriodic` 使用简单的 PID 控制循环，驱动升降机构到距地面 30 英寸的 setpoint。

```

@Override
public void teleopPeriodic() {
    if (m_joystick.getTrigger()) {
        // Here, we set the constant setpoint of 0.75 meters.
        m_elevator.reachGoal(Constants.kSetpointMeters);
    } else {
        // Otherwise, we update the setpoint to 0.
        m_elevator.reachGoal(0.0);
    }
}

```

```

public void reachGoal(double goal) {
    m_controller.setGoal(goal);

    // With the setpoint value we run PID control like normal
    double pidOutput = m_controller.calculate(m_encoder.getDistance());
    double feedforwardOutput = m_feedforward.calculate(m_controller.getSetpoint().velocity);
    m_motor.setVoltage(pidOutput + feedforwardOutput);
}

```

Next, `simulationPeriodic` uses the voltage applied to the motor to update the simulated position of the elevator. We use `simulationPeriodic` because it runs periodically only for simulated robots. This means that our simulation code will not be run on a real robot.

接下来，`simulationPeriodic` 使用施加在电机上的电压更新升降机构的仿真位置。使用 `simulationPeriodic` 是因为它只会对仿真机器人周期性运行。这意味着仿真代码不会在实体机器人上运行。

Note / 说明

Classes inheriting from command-based's `Subsystem` can override the inherited `simulationPeriodic()` method. Other classes will need their simulation update methods called from Robot's `simulationPeriodic`.

继承 Command-Based `Subsystem` 的类可以重写继承而来的 `simulationPeriodic()` 方法。其他类需要从 Robot 的 `simulationPeriodic` 中调用各自的仿真更新方法。

Finally, the simulated encoder's distance reading is set using the simulated elevator's position, and the robot's battery voltage is set using the estimated current drawn by the elevator.

最后，使用仿真升降机构的位置设置仿真编码器的距离读数，并根据升降机构预计消耗的电流设置机器人电池电压。

```

public void simulationPeriodic() {
    // In this method, we update our simulation of what our elevator is doing
    // First, we set our "inputs" (voltages)

```

```
m_elevatorSim.setInput(m_motorSim.getSpeed() * RobotController.getBatteryVoltage());

// Next, we update it. The standard loop time is 20ms.
m_elevatorSim.update(0.020);

// Finally, we set our simulated encoder's readings and simulated battery voltage
m_encoderSim.setDistance(m_elevatorSim.getPositionMeters());
// SimBattery estimates loaded battery voltages
RoboRioSim.setVinVoltage(
    BatterySim.calculateDefaultBatteryLoadedVoltage(m_elevatorSim.getCurrentDrawAmps()));
}
```

Device Simulation / 设备仿真

WPILib provides a way to manage simulation device data in the form of the SimDevice API.

WPILib 提供 SimDevice API，用于管理仿真设备数据。

Simulating Core WPILib Device Classes / 仿真 WPILib 核心设备类

Core WPILib device classes (i.e Encoder, Ultrasonic, etc.) have simulation classes named EncoderSim, UltrasonicSim, and so on. These classes allow interactions with the device data that wouldn't be possible or valid outside of simulation. Constructing them outside of simulation likely won't interfere with your code, but calling their functions and the like is undefined behavior - in the best case they will do nothing, worse cases might crash your code! Place functional simulation code in simulation-only functions (such as `simulationPeriodic()`) or wrap them with `RobotBase.isReal()` checks.

WPILib 的核心设备类（例如 Encoder、Ultrasonic 等）有对应的仿真类，分别称为 EncoderSim、UltrasonicSim 等。这些类允许与设备数据进行一些在仿真之外不可能或无效的交互。在仿真之外构造它们通常不会干扰代码，但调用其函数等操作的行为未定义：轻则没有效果，严重时可能导致代码崩溃！应将具有实际功能的仿真代码放在仅仿真时运行的函数中（例如 `simulationPeriodic()`），或使用 `RobotBase.isReal()` 检查将其包裹起来。

Note / 说明

This example will use the EncoderSim class as an example. Use of other simulation classes will be almost identical.

本示例使用 EncoderSim 类作为示例。其他仿真类的用法几乎相同。

Creating Simulation Device objects / 创建设备仿真对象

Simulation device object can be constructed in two ways:

可以通过两种方式构造仿真设备对象：

- a constructor that accepts the regular hardware object.
- 使用接收常规硬件对象的构造函数。
- a constructor or factory method that accepts the port/index/channel number that the device is connected to. These would be the same number that was used to construct the regular hardware object. This is especially useful for [unit testing](#).
- 使用接收设备所连接端口、索引或通道编号的构造函数或工厂方法。这应与构造常规硬件对象时使用的编号相同。对于[单元测试](#)，这种方式尤其有用。

```
// create a real encoder object on DIO 2,3
Encoder encoder = new Encoder(2, 3);
// create a sim controller for the encoder
EncoderSim simEncoder = new EncoderSim(encoder);
```

Reading and Writing Device Data / 读取和写入设备数据

Each simulation class has getter (getXxx()) and setter (setXxx(value)) functions for each field Xxx. The getter functions will return the same as the getter of the regular device class.

每个仿真类都为每个字段 Xxx 提供 getter (getXxx()) 和 setter (setXxx(value)) 函数。getter 函数返回的值与常规设备类的 getter 相同。

```
simEncoder.setCount(100);
encoder.getCount(); // 100
simEncoder.getCount(); // 100
```

Registering Callbacks / 注册回调

In addition to the getters and setters, each field also has a registerXxxCallback() function that registers a callback to be run whenever the field value changes and returns a CallbackStore object. The callbacks accept a string parameter of the name of the field and a HALValue object containing the new value. Before retrieving values from a HALValue, check the type of value contained. Possible types are HALValue.kBoolean, HALValue.kDouble, HALValue.kEnum, HALValue.kInt, HALValue.kLong.

除 getter 和 setter 外，每个字段还有一个 registerXxxCallback() 函数，用于注册字段值发生变化时运行的回调，并返回一个 CallbackStore 对象。回调接收一个表示字段名称的字符串参数，以及一个包含新值的 HALValue 对象。从 HALValue 读取值之前，应检查其包含的值的类型。可能的类型包括 HALValue.kBoolean、HALValue.kDouble、HALValue.kEnum、HALValue.kInt 和 HALValue.kLong。

In Java, call close() on the CallbackStore object to cancel the callback. Keep a reference to the object so it doesn't get garbage-collected - otherwise the callback will be canceled by GC. To provide arbitrary data to the callback, capture it in the lambda or use a method reference.

在 Java 中，调用 CallbackStore 对象的 close() 可取消回调。应保留对该对象的引用，避免它被垃圾回收；否则 GC 会取消回调。要向回调提供任意数据，可在 lambda 中捕获该数据，或使用方法引用。

Warning / 警告

Attempting to retrieve a value of a type from a HALValue containing a different type is undefined behavior.

如果从 HALValue 中尝试读取与其实际所含类型不同的值，其行为未定义。

```
NotifyCallback callback = (String name, HALValue value) -> {
    if (value.getType() == HALValue.kInt) {
        System.out.println("Value of " + name + " is " + value.getInt());
    }
}
CallbackStore store = simEncoder.registerCountCallback(callback);
store.close(); // cancel the callback
```

Simulating Other Devices - The SimDeviceSim Class / 仿真其他设备：SimDeviceSim 类

Note / 说明

Vendors might implement their connection to the SimDevice API slightly different than described here. They might also provide a simulation class specific for their device class. See your vendor's documentation for more information as to what they support and how.

供应商与 SimDevice API 的集成方式可能与这里所述略有不同。他们也可能为自己的设备类提供专用的仿真类。有关支持范围和使用方法，请查阅相应供应商的文档。

The SimDeviceSim (**not** SimDevice!) class is a general device simulation object for devices that aren't core WPILib devices and therefore don't have specific simulation classes - such as vendor devices. These devices will show up in the Other Devices tab of the [SimGUI](#).

SimDeviceSim (**不是** SimDevice!) 是一种通用设备仿真对象，适用于不属于 WPILib 核心设备、因此没有专用仿真类的设备，例如供应商设备。这些设备会显示在 [SimGUI](#) 的 Other Devices 标签中。

The SimDeviceSim object is created using a string key identical to the key the vendor used to construct the underlying SimDevice in their device class. This key is the one that the device shows up with in the Other Devices tab, and is typically of the form Prefix:Device Name[index]. If the key contains ports/index/channel numbers, they can be passed as separate arguments to the SimDeviceSim constructor. The key contains a prefix that is hidden by default in the SimGUI, it can be shown by selecting the Show prefix option. Not including this prefix in the key passed to SimDeviceSim will not match the device!

创建 SimDeviceSim 对象时，使用的字符串键必须与供应商在其设备类中构造底层 SimDevice 时使用的键完全相同。设备会以这个键显示在 Other Devices 标签中；键通常采用 Prefix:Device Name[index] 形式。如果键包含端口、索引或通道编号，也可以将这些编号作为单独参数传给 SimDeviceSim 构造函数。键中还包含一个前缀，SimGUI 默认将其隐藏；选择 Show prefix 可显示该前缀。如果传给 SimDeviceSim 的键未包含此前缀，就无法匹配到设备！

```
SimDeviceSim device = new SimDeviceSim(deviceKey, index);
```

Once we have the SimDeviceSim, we can get SimValue objects representing the device's fields. Type-specific SimDouble, SimInt, SimLong, SimBoolean, and SimEnum subclasses also exist, and should be used instead of the type-unsafe SimValue class. These are constructed from the SimDeviceSim using a string key identical to the one the vendor used to define the field. This key is the one the field appears as in the SimGUI. Attempting to retrieve a SimValue object outside of simulation or when either the device or field keys are unmatched will return null - this can cause NullPointerException in Java.

得到 SimDeviceSim 后，可以获取表示设备字段的 SimValue 对象。此外还有特定类型的 SimDouble、SimInt、SimLong、SimBoolean 和 SimEnum 子类；应优先使用它们，而不是类型不安全的 SimValue 类。使用与供应商定义字段时相同的字符串键，可从 SimDeviceSim 构造这些对象。字段在 SimGUI 中也以这个键显示。如果在仿真之外获取 SimValue 对象，或者设备键或字段键不匹配，就会返回 null，这可能在 Java 中导致 NullPointerException。

```
SimDouble field = device.getDouble(fieldKey);
field.get();
field.set(value);
```

Drivetrain Simulation Overview / 底盘仿真概览

Note / 说明

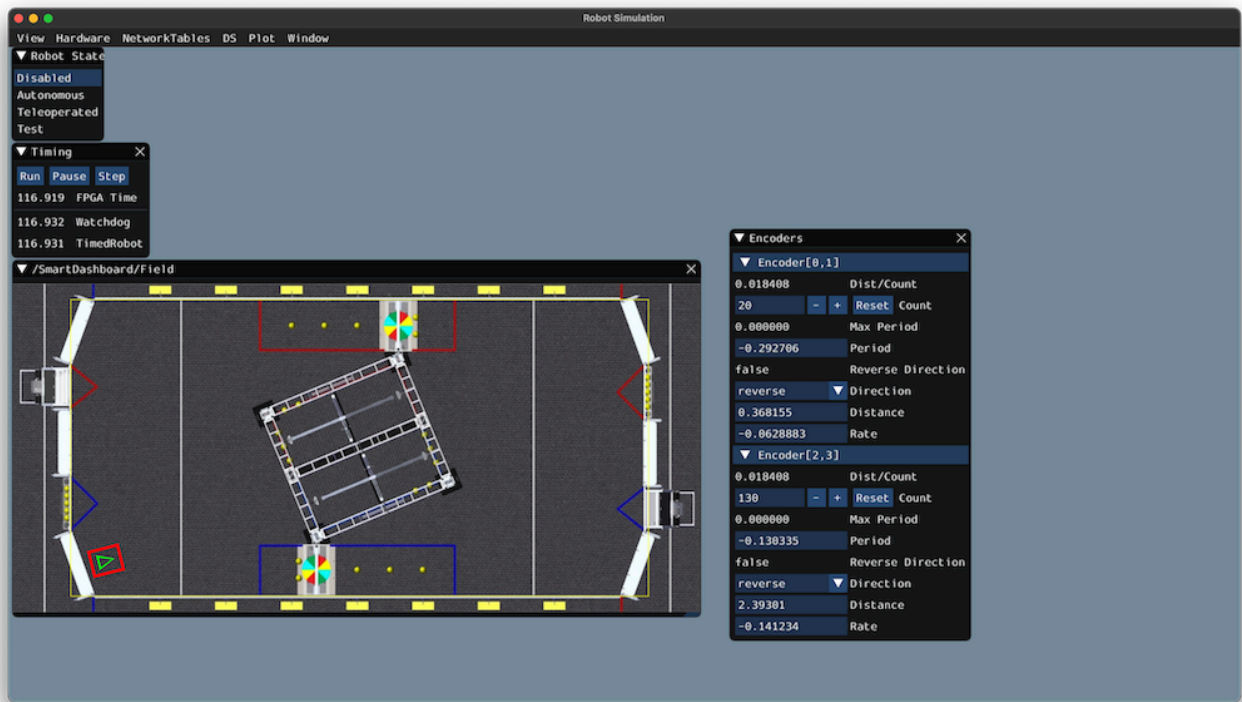
The code in this tutorial does not use any specific framework (i.e. command-based vs. simple data flow); however, guidance will be provided in certain areas for how to best implement certain pieces of code in specific framework types.

本教程的代码不依赖特定框架（例如 Command-Based 或简单数据流）；不过，在某些部分会说明如何针对特定框架类型更好地实现代码。

The goal of this tutorial is to provide guidance on implementing simulation capabilities for a differential-drivetrain robot. By the end of this tutorial, you should be able to:

本教程旨在指导你为差速底盘机器人实现仿真功能。完成本教程后，你应能够：

1. Understand the basic underlying concepts behind the WPILib simulation framework.
1. 理解 WPILib 仿真框架背后的基本概念。
2. Create a drivetrain simulation model using your robot's physical parameters.
2. 使用机器人的物理参数创建底盘仿真模型。
3. Use the simulation model to predict how your real robot will move given specific voltage inputs.
3. 使用仿真模型预测实体机器人在特定电压输入下的运动方式。
4. Tune feedback constants and squash common bugs (e.g. motor inversion) before having access to physical hardware.
4. 在接触实体硬件之前，调校反馈常量并排除电机反向等常见错误。
5. Use the Simulation GUI to visualize robot movement on a virtual field.
5. 使用仿真 GUI 在虚拟场地上查看机器人运动。



Why Simulate a Drivetrain? / 为什么要仿真底盘?

The drivetrain of a robot is one of the most important mechanisms on the robot – therefore, it is important to ensure that the software powering your drivetrain is as robust as possible. By being able to simulate how a physical drivetrain responds, you can get a head start on writing quality software before you have access to the physical hardware. With the simulation framework, you can verify not only basic functionality, like making sure that the inversions on motors and encoders are correct, but also advanced capabilities such as verifying accuracy of path following.

机器人的底盘是最重要的机构之一，因此确保驱动底盘的软件尽可能可靠十分重要。通过仿真实体底盘的响应，可以在拿到实体硬件之前就开始编写高质量的软件。借助仿真框架，不仅可以验证电机和编码器方向是否正确等基础功能，还可以验证路径跟踪精度等高级功能。

Step 1: Creating Simulated Instances of Hardware / 第 1 步：创建硬件的仿真实例

The WPILib simulation framework contains several XXXSim classes, where XXX represents physical hardware such as encoders or gyroscopes. These simulation classes can be used to set positions and velocities (for encoders) and angles (for gyroscopes) from a model of your drivetrain. See [the Device Simulation article](#) for more info about these simulation hardware classes and simulation of vendor devices.

WPILib 仿真框架包含多个 XXXSim 类，其中 XXX 表示编码器、陀螺仪等实体硬件。这些仿真类可根据底盘模型设置编码器的位置和速度，以及陀螺仪的角度。有关这些仿真硬件类及供应商设备仿真的更多信息，请参阅[设备仿真文章](#)。

Note / 说明

Simulation objects associated with a particular subsystem should live in that subsystem. An example of this is in the StateSpaceDriveSimulation ([Java](#)) example.

与特定 subsystem 关联的仿真对象应位于该 subsystem 中。StateSpaceDriveSimulation 示例 ([Java](#)) 就是一个例子。

Simulating Encoders / 仿真编码器

The EncoderSim class allows users to set encoder positions and velocities on a given Encoder object. When running on real hardware, the Encoder class interacts with real sensors to count revolutions (and convert them to distance units automatically if configured to do so); however, in simulation there are no such measurements to make. The EncoderSim class can accept these simulated readings from a model of your drivetrain.

EncoderSim 类允许用户为指定的 Encoder 对象设置编码器位置和速度。在实体硬件上运行时，Encoder 类会与真实传感器交互，统计旋转圈数（如果已配置，还会自动转换为距离单位）；而在仿真中并不存在可供测量的真实运动。EncoderSim 类可以接收来自底盘模型的仿真读数。

Note / 说明

It is not possible to simulate encoders that are directly connected to CAN motor controllers using WPILib classes. For more information about your specific motor controller, please read your vendor's documentation.

无法使用 WPILib 类仿真直接连接到 CAN 电机控制器的编码器。有关特定电机控制器的更多信息，请阅读供应商文档。

```
// These represent our regular encoder objects, which we would
// create to use on a real robot.
private Encoder m_leftEncoder = new Encoder(0, 1);
private Encoder m_rightEncoder = new Encoder(2, 3);
// These are our EncoderSim objects, which we will only use in
// simulation. However, you do not need to comment out these
```

```
// declarations when you are deploying code to the roboRIO.  
private EncoderSim m_leftEncoderSim = new EncoderSim(m_leftEncoder);  
private EncoderSim m_rightEncoderSim = new EncoderSim(m_rightEncoder);
```

Simulating Gyroscopes / 仿真陀螺仪

Similar to the EncoderSim class, simulated gyroscope classes also exist for commonly used WPILib gyros – AnalogGyroSim and ADXRS450_GyroSim. These are also constructed in the same manner.

与 EncoderSim 类似，WPILib 常用陀螺仪也有仿真类：AnalogGyroSim 和 ADXRS450_GyroSim。它们的构造方式也相同。

Note / 说明

It is not possible to simulate certain vendor gyros (i.e. Pigeon [IMU](#) and NavX) using WPILib classes. Please read the respective vendors' documentation for information on their simulation support.

无法使用 WPILib 类仿真某些供应商陀螺仪（例如 Pigeon [IMU](#) 和 NavX）。有关它们的仿真支持，请查阅相应供应商的文档。

```
// Create our gyro object like we would on a real robot.  
private AnalogGyro m_gyro = new AnalogGyro(1);  
// Create the simulated gyro object, used for setting the gyro  
// angle. Like EncoderSim, this does not need to be commented out  
// when deploying code to the roboRIO.  
private AnalogGyroSim m_gyroSim = new AnalogGyroSim(m_gyro);
```

Step 2: Creating a Drivetrain Model / 第 2 步：创建底盘模型

In order to accurately determine how your physical drivetrain will respond to given motor voltage inputs, an accurate model of your drivetrain must be created. This model is usually created by measuring various physical parameters of your real robot. In WPILib, this drivetrain simulation model is represented by the `DifferentialDrivetrainSim` class.

为了准确判断实体底盘在给定电机电压输入下的响应，必须建立准确的底盘模型。通常通过测量实体机器人的各项物理参数来建立这个模型。在 WPILib 中，底盘仿真模型由 `DifferentialDrivetrainSim` 类表示。

Creating a DifferentialDrivetrainSim from Physical Measurements / 根据物理测量值创建 DifferentialDrivetrainSim

One way to creating a `DifferentialDrivetrainSim` instance is by using physical measurements of the drivetrain and robot – either obtained through [CAD](#) software or real-world measurements (the latter will usually yield better results as it will more closely match reality). This constructor takes the following parameters:

创建 `DifferentialDrivetrainSim` 实例的一种方法，是使用底盘和机器人的物理测量值。这些值可以来自 [CAD](#) 软件，也可以来自实际测量；后者通常更贴近真实情况，因此效果更好。此构造函数接受以下参数：

- The type and number of motors on one side of the drivetrain.
- 底盘一侧电机的类型和数量。
- The gear ratio between the motors and the wheels as output [torque](#) over input [torque](#) (this number is usually greater than 1 for drivetrains).
- 电机与车轮之间以输出[扭矩](#)除以输入[扭矩](#)表示的齿轮比（对于底盘，这个数值通常大于 1）。
- The moment of inertia of the drivetrain (this can be obtained from a [CAD](#) model of your drivetrain. Usually, this is between 3 and 8 (kg m^2)).
- 底盘的转动惯量（可从底盘的 [CAD](#) 模型获取。通常为 3 到 8 (kg m^2))。
- The mass of the drivetrain (it is recommended to use the mass of the entire robot itself, as it will more accurately model the acceleration characteristics of your robot for trajectory tracking).
- 底盘质量（建议使用整台机器人的质量，因为这样能更准确地模拟机器人在轨迹跟踪时的加速特性）。
- The radius of the drive wheels.
- 驱动轮的半径。
- The track width (distance between left and right wheels).
- 轮距（左右车轮之间的距离）。
- Standard deviations of measurement noise: this represents how much measurement noise you expect from your real sensors. The measurement noise is an array with 7 elements, with each element representing the standard deviation of measurement noise in x, y, heading, left velocity, right velocity, left position, and right position respectively. This option can be set to `null` in Java if measurement noise is not desirable.
- 测量噪声的标准差：表示预计真实传感器会产生多大的测量噪声。测量噪声是一个包含 7 个元素的数组，各元素依次表示 x、y、朝向、左侧速度、右侧速度、左侧位置和右侧位置的测量噪声标准差。如果不需要测量噪声，在 Java 中可将此选项设为 `null`。

You can calculate the measurement noise of your sensors by taking multiple data points of the state you are trying to measure and calculating the standard deviation using a tool like Python. For example, to calculate the standard deviation in your encoders' velocity estimate, you can move your robot at a constant velocity, take

multiple measurements, and calculate their standard deviation from the known mean. If this process is too tedious, the values used in the example below should be a good representation of average noise from encoders.

可以针对要测量的状态采集多个传感器数据点，并使用 Python 等工具计算标准差，从而估计传感器的测量噪声。例如，要计算编码器速度估计的标准差，可以让机器人以恒定速度运动，多次测量，并根据已知平均值计算标准差。如果这一过程过于繁琐，下面示例中的数值应能较好地代表编码器的平均噪声。

Note / 说明

The standard deviation of the noise for a measurement has the same units as that measurement. For example, the standard deviation of the velocity noise has units of m/s.

某项测量的噪声标准差与该测量使用相同单位。例如，速度噪声的标准差使用 m/s。

Note / 说明

It is very important to use SI units (i.e. meters and radians) when passing parameters in Java.

在 Java 中传入参数时，使用 SI 单位（即米和弧度）非常重要。

```
// Create the simulation model of our drivetrain.
DifferentialDrivetrainSim m_driveSim = new DifferentialDrivetrainSim(
    DCMotor.getNEO(2),          // 2 NEO motors on each side of the drivetrain.
    7.29,                       // 7.29:1 gearing reduction.
    7.5,                        // MOI of 7.5 kg m^2 (from CAD model).
    60.0,                       // The mass of the robot is 60 kg.
    Units.inchesToMeters(3),    // The robot uses 3" radius wheels.
    0.7112,                    // The track width is 0.7112 meters.
    // The standard deviations for measurement noise:
    // x and y:                  0.001 m
    // heading:                  0.001 rad
    // l and r velocity: 0.1    m/s
    // l and r position: 0.005 m
    VecBuilder.fill(0.001, 0.001, 0.001, 0.1, 0.1, 0.005, 0.005));
```

Creating a DifferentialDrivetrainSim from SysId Gains / 根据 SysId 增益创建 DifferentialDrivetrainSim

You can also use the gains produced by [System Identification](#), which you may have performed as part of setting up the trajectory tracking workflow outlined [here](#) to create a simulation model of your drivetrain and often yield results closer to real-world behavior than the method above.

还可以使用[系统辨识](#)得到的增益来创建底盘仿真模型。你可能已经在按照[这里](#)所述的轨迹跟踪流程进行设置时完成了系统辨识；与上述方法相比，这些增益通常能产生更接近实体机器人行为的结果。

Important / 重要

You must need two sets of K_v and K_a gains from the identification tool – one from straight-line motion and the other from rotating in place. We will refer to these two sets of gains as linear and angular gains respectively.

需要从辨识工具获取两组 K_v 和 K_a 增益：一组来自直线运动，另一组来自原地旋转。下文分别将它们称为线性增益和角向增益。

This constructor takes the following parameters:

此构造函数接受以下参数：

- A linear system representing the drivetrain – this can be created using the identification gains.
- 表示底盘的线性系统；可利用辨识增益创建。
- The track width (distance between the left and right wheels).
- 轮距（左右车轮之间的距离）。
- The type and number of motors on one side of the drivetrain.
- 底盘一侧电机的类型和数量。
- The gear ratio between the motors and the wheels as output [torque](#) over input [torque](#) (this number is usually greater than 1 for drivetrains).
- 电机与车轮之间以输出[扭矩](#)除以输入[扭矩](#)表示的齿轮比（对于底盘，这个数值通常大于 1）。
- The radius of the drive wheels.
- 驱动轮的半径。
- Standard deviations of measurement noise: this represents how much measurement noise you expect from your real sensors. The measurement noise is an array with 7 elements, with each element representing the standard deviation of measurement noise in x, y, heading, left velocity, right velocity, left position, and right position respectively. This option can be set to null in Java if measurement noise is not desirable.
- 测量噪声的标准差：表示预计真实传感器会产生多大的测量噪声。测量噪声是一个包含 7 个元素的数组，各元素依次表示 x、y、朝向、左侧速度、右侧速度、左侧位置和右侧位置的测量噪声标准差。如果不需要测量噪声，在 Java 中可将此选项设为 null。

You can calculate the measurement noise of your sensors by taking multiple data points of the state you are trying to measure and calculating the standard deviation using a tool like Python. For example, to calculate the standard deviation in your encoders' velocity estimate, you can move your robot at a constant velocity, take multiple measurements, and calculate their standard deviation from the known mean. If this process is too tedious, the values used in the example below should be a good representation of average noise from encoders.

可以针对要测量的状态采集多个传感器数据点，并使用 Python 等工具计算标准差，从而估计传感器的测量噪声。例如，要计算编码器速度估计的标准差，可以让机器人以恒定速度运动，多次测量，并根据已知平均值计算标准差。如果这一过程过于繁琐，下面示例中的数值应能较好地代表编码器的平均噪声。

Note / 说明

The standard deviation of the noise for a measurement has the same units as that measurement. For example, the standard deviation of the velocity noise has units of m/s.

某项测量的噪声标准差与该测量使用相同单位。例如，速度噪声的标准差使用 m/s。

Note / 说明

It is very important to use SI units (i.e. meters and radians) when passing parameters in Java.

在 Java 中传入参数时，使用 SI 单位（即米和弧度）非常重要。

```
// Create our feedforward gain constants (from the identification
// tool)
static final double KvLinear = 1.98;
static final double KaLinear = 0.2;
static final double KvAngular = 1.5;
static final double KaAngular = 0.3;
// Create the simulation model of our drivetrain.
private DifferentialDrivetrainSim m_driveSim = new DifferentialDrivetrainSim(
// Create a linear system from our identification gains.
    LinearSystemId.identifyDrivetrainSystem(KvLinear, KaLinear, KvAngular, KaAngular),
    DCMotor.getNEO(2),          // 2 NEO motors on each side of the drivetrain.
    7.29,                       // 7.29:1 gearing reduction.
    0.7112,                     // The track width is 0.7112 meters.
    Units.inchesToMeters(3),    // The robot uses 3" radius wheels.
    // The standard deviations for measurement noise:
    // x and y:                  0.001 m
    // heading:                  0.001 rad
    // l and r velocity: 0.1    m/s
    // l and r position: 0.005 m
    VecBuilder.fill(0.001, 0.001, 0.001, 0.1, 0.1, 0.005, 0.005));
```

Creating a DifferentialDrivetrainSim of the KoP Chassis / 为 KoP 底盘创建 DifferentialDrivetrainSim

The `DifferentialDrivetrainSim` class also has a static `createKitbotSim()` method that can create an instance of the `DifferentialDrivetrainSim` using the standard Kit of Parts Chassis parameters. This method takes 5 arguments, two of which are optional:

`DifferentialDrivetrainSim` 类还提供静态方法 `createKitbotSim()`，可以使用标准 Kit of Parts 底盘的参数创建 `DifferentialDrivetrainSim` 实例。该方法接受 5 个参数，其中 2 个是可选参数：

- The type and number of motors on one side of the drivetrain.
- 底盘一侧电机的类型和数量。
- The gear ratio between the motors and the wheels as output [torque](#) over input [torque](#) (this number is usually greater than 1 for drivetrains).
- 电机与车轮之间以输出[扭矩](#)除以输入[扭矩](#)表示的齿轮比（对于底盘，这个数值通常大于 1）。
- The diameter of the wheels installed on the drivetrain.
- 底盘所装车轮的直径。
- The moment of inertia of the drive base (optional).
- 驱动底盘的转动惯量（可选）。

- Standard deviations of measurement noise: this represents how much measurement noise you expect from your real sensors. The measurement noise is an array with 7 elements, with each element representing the standard deviation of measurement noise in x, y, heading, left velocity, right velocity, left position, and right position respectively. This option can be set to null in Java if measurement noise is not desirable.
- 测量噪声的标准差：表示预计真实传感器会产生多大的测量噪声。测量噪声是一个包含 7 个元素的数组，各元素依次表示 x、y、朝向、左侧速度、右侧速度、左侧位置和右侧位置的测量噪声标准差。如果不需要测量噪声，在 Java 中可将此选项设为 null。

You can calculate the measurement noise of your sensors by taking multiple data points of the state you are trying to measure and calculating the standard deviation using a tool like Python. For example, to calculate the standard deviation in your encoders' velocity estimate, you can move your robot at a constant velocity, take multiple measurements, and calculate their standard deviation from the known mean. If this process is too tedious, the values used in the example below should be a good representation of average noise from encoders.

可以针对要测量的状态采集多个传感器数据点，并使用 Python 等工具计算标准差，从而估计传感器的测量噪声。例如，要计算编码器速度估计的标准差，可以让机器人以恒定速度运动，多次测量，并根据已知平均值计算标准差。如果这一过程过于繁琐，下面示例中的数值应能较好地代表编码器的平均噪声。

Note / 说明

The standard deviation of the noise for a measurement has the same units as that measurement. For example, the standard deviation of the velocity noise has units of m/s.

某项测量的噪声标准差与该测量使用相同单位。例如，速度噪声的标准差使用 m/s。

Note / 说明

It is very important to use SI units (i.e. meters and radians) when passing parameters in Java.

在 Java 中传入参数时，使用 SI 单位（即米和弧度）非常重要。

```
private DifferentialDriveTrainSim m_driveSim = DifferentialDriveTrainSim.createKitbotSim(
    KitbotMotor.kDualCIMPerSide, // 2 CIMs per side.
    KitbotGearing.k10p71,        // 10.71:1
    KitbotWheelSize.kSixInch,    // 6" diameter wheels.
    null                          // No measurement noise.
);
```

Note / 说明

You can use the KitbotMotor, KitbotGearing, and KitbotWheelSize enum to get commonly used configurations of the Kit of Parts Chassis.

可以使用 KitbotMotor、KitbotGearing 和 KitbotWheelSize 枚举获取 Kit of Parts 底盘的常见配置。

Important / 重要

Constructing your `DifferentialDriveTrainSim` instance in this way is just an approximation and is intended to get teams quickly up and running with simulation. Using empirical values measured from your physical robot will always yield more accurate results.

以这种方式构造 `DifferentialDriveTrainSim` 实例只是一种近似，目的是帮助队伍快速开始使用仿真。使用在实体机器人上测得的经验数值，始终会得到更准确的结果。

Step 3: Updating the Drivetrain Model / 第 3 步：更新底盘模型

Now that the drivetrain model has been made, it needs to be updated periodically with the latest motor voltage commands. It is recommended to do this step in a separate `simulationPeriodic()` method inside your subsystem and only call this method in simulation.

创建底盘模型后，需要定期使用最新的电机电压命令更新它。建议在 `subsystem` 中单独使用 `simulationPeriodic()` 方法完成此步骤，并且只在仿真中调用该方法。

Note / 说明

If you are using the command-based framework, every subsystem that extends `SubsystemBase` has a `simulationPeriodic()` which can be overridden. This method is automatically run only during simulation. If you are not using the command-based library, make sure you call your simulation method inside the overridden `simulationPeriodic()` of the main `Robot` class. These periodic methods are also automatically called only during simulation.

如果使用 Command-Based 框架，每个继承自 `SubsystemBase` 的 `subsystem` 都有一个可重写的 `simulationPeriodic()`。此方法只在仿真时自动运行。如果不使用 Command-Based 库，请确保从主 `Robot` 类重写的 `simulationPeriodic()` 中调用仿真方法。这些周期方法同样只在仿真时自动运行。

There are three main steps to updating the model:

更新模型主要分为三个步骤：

1. Set the [input](#) of the drivetrain model. These are the motor voltages from the two sides of the drivetrain.
1. 设置底盘模型的[输入](#)，即底盘两侧电机的电压。
2. Advance the model forward in time by the nominal periodic timestep (Usually 20 ms). This updates all of the drivetrain's states (i.e. pose, encoder positions and velocities) as if 20 ms had passed.
2. 按标称周期时间步推进模型（通常为 20 ms）。这会更新底盘的所有状态（例如位姿（pose）、编码器位置和速度），相当于经过了 20 ms。
3. Update simulated sensors with new positions, velocities, and angles to use in other places.
3. 用新的位置、速度和角度更新仿真传感器，供其他部分使用。

```
private PWMSparkMax m_leftMotor = new PWMSparkMax(0);
private PWMSparkMax m_rightMotor = new PWMSparkMax(1);
public Drivetrain() {
    ...
    m_leftEncoder.setDistancePerPulse(2 * Math.PI * kWheelRadius / kEncoderResolution);
    m_rightEncoder.setDistancePerPulse(2 * Math.PI * kWheelRadius / kEncoderResolution);
}
public void simulationPeriodic() {
    // Set the inputs to the system. Note that we need to convert
    // the [-1, 1] PWM signal to voltage by multiplying it by the
    // robot controller voltage.
    m_driveSim.setInputs(m_leftMotor.get() * RobotController.getInputVoltage(),
                        m_rightMotor.get() * RobotController.getInputVoltage());
    // Advance the model by 20 ms. Note that if you are running this
    // subsystem in a separate thread or have changed the nominal timestep
    // of TimedRobot, this value needs to match it.
```

```
m_driveSim.update(0.02);
// Update all of our sensors.
m_leftEncoderSim.setDistance(m_driveSim.getLeftPositionMeters());
m_leftEncoderSim.setRate(m_driveSim.getLeftVelocityMetersPerSecond());
m_rightEncoderSim.setDistance(m_driveSim.getRightPositionMeters());
m_rightEncoderSim.setRate(m_driveSim.getRightVelocityMetersPerSecond());
m_gyroSim.setAngle(-m_driveSim.getHeading().getDegrees());
}
```

Important / 重要

If the right side of your drivetrain is inverted, you MUST negate the right voltage in the `setInputs()` call to ensure that positive voltages correspond to forward movement.

如果底盘右侧反向，则在调用 `setInputs()` 时**必须**将右侧电压取反，确保正电压对应前进运动。

Important / 重要

Because the drivetrain simulator model returns positions and velocities in meters and m/s respectively, these must be converted to encoder ticks and ticks/s when calling `setDistance()` and `setRate()`. Alternatively, you can configure `setDistancePerPulse()` on the encoders to have the Encoder object take care of this automatically – this is the approach that is taken in the example above.

由于底盘仿真模型返回的位置和速度分别以米和 m/s 为单位，因此在调用 `setDistance()` 和 `setRate()` 时，必须将其转换为编码器脉冲数和脉冲数/s。也可以在编码器上配置 `setDistancePerPulse()`，由 Encoder 对象自动完成转换；上述示例采用的就是这种方法。

Now that the simulated encoder positions, velocities, and gyroscope angles have been set, you can call `m_leftEncoder.getDistance()`, etc. in your robot code as normal and it will behave exactly like it would on a real robot. This involves performing odometry calculations, running velocity PID feedback loops for trajectory tracking, etc.

设置仿真编码器的位置和速度以及陀螺仪角度后，便可以像在实体机器人上一样，从机器人代码中正常调用 `m_leftEncoder.getDistance()` 等方法。这包括执行里程计（odometry）计算、运行用于轨迹跟踪的速度 PID 反馈循环等。

Step 4: Updating Odometry and Visualizing Robot Position / 第 4 步：更新里程计并可视化机器人位置

Now that the simulated encoder positions, velocities, and gyro angles are being updated with accurate information periodically, this data can be used to update the pose of the robot in a periodic loop (such as the `periodic()` method in a `Subsystem`). In simulation, the periodic loop will use simulated encoder and gyro readings to update odometry whereas on the real robot, the same code will use real readings from physical hardware.

现在，仿真编码器的位置、速度以及陀螺仪角度都已定期用准确的信息更新；可以在周期循环中使用这些数据更新机器人的位姿（pose），例如在 `Subsystem` 的 `periodic()` 方法中。在仿真中，该周期循环使用仿真编码器和陀螺仪读数更新里程计（odometry）；在实体机器人上，同一份代码则使用实体硬件的真实读数。

Note / 说明

For more information on using odometry, see [this document](#).

有关里程计的更多信息，请参阅[这篇文档](#)。

Robot Pose Visualization / 机器人位姿可视化

The robot pose can be visualized on the Simulator GUI (during simulation) or on a dashboard such as Glass (on a real robot) by sending the odometry pose over a `Field2d` object. A `Field2d` can be trivially constructed without any constructor arguments:

可以通过 `Field2d` 对象发送里程计位姿，在仿真期间的仿真 GUI 或实体机器人上的 Glass 等仪表板中可视化机器人位姿。构造 `Field2d` 时无需传入任何构造参数：

```
private Field2d m_field = new Field2d();
```

This `Field2d` instance must then be sent over `NetworkTables`. The best place to do this is in the constructor of your subsystem.

接下来，必须通过 `NetworkTables` 发送这个 `Field2d` 实例。最合适的位置是 `Subsystem` 的构造函数。

```
public Drivetrain() {  
    ...  
    SmartDashboard.putData("Field", m_field);  
}
```

Note / 说明

The `Field2d` instance can also be sent using a lower-level `NetworkTables` API or using the [Shuffleboard API](#).

也可以使用底层 `NetworkTables` API 或 [Shuffleboard API](#) 发送 `Field2d` 实例。

Finally, the pose from your odometry must be updated periodically into the `Field2d` object. Remember that this should be in a general `periodic()` method i.e. one that runs both during simulation and during real robot operation.

最后，必须定期将里程计得到的位姿更新到 `Field2d` 对象中。请记住，应在通用的 `periodic()` 方法中完成此操作，也就是在仿真和实体机器人运行时都会执行的方法。

```
public void periodic() {
    ...
    // This will get the simulated sensor readings that we set
    // in the previous article while in simulation, but will use
    // real values on the robot itself.
    m_odometry.update(m_gyro.getRotation2d(),
                     m_leftEncoder.getDistance(),
                     m_rightEncoder.getDistance());
    m_field.setRobotPose(m_odometry.getPoseMeters());
}
```

Important / 重要

It is important that this code is placed in a regular `periodic()` method – one that is called periodically regardless of mode of operation. If you are using the command-based library, this method already exists. If not, you are responsible for calling this method periodically from the main `Robot` class.

此代码必须放在普通的 `periodic()` 方法中，即无论运行模式如何都会周期性调用的方法。如果使用 `Command-Based` 库，该方法已存在；否则，需要负责从主 `Robot` 类定期调用此方法。

Note / 说明

At this point we have covered all of the code changes required to run your code. You should head to the [Simulation User Interface page](#) for more info on how to run the simulation and the [Field2d Widget page](#) to add the field that your simulated robot will run on to the GUI.

至此，运行代码所需的全部代码修改都已介绍完毕。有关如何运行仿真，请参阅[仿真用户界面页面](#)；有关如何在 GUI 中加入仿真机器人运行的场地，请参阅[Field2d Widget 页面](#)。

Unit Testing / 单元测试

Unit testing is a method of testing code by dividing the code into the smallest “units” possible and testing each unit. In robot code, this can mean testing the code for each subsystem individually. There are many unit testing frameworks for most languages. Java robot projects have [JUnit 5](#) available by default.

单元测试是一种将代码划分为尽可能小的“单元”，并分别测试各单元的方法。在机器人代码中，这可能意味着分别测试各个 subsystem 的代码。大多数语言都有多种单元测试框架。Java 机器人项目默认提供 [JUnit 5](#)。

Writing Testable Code / 编写可测试的代码

Note / 说明

This example can be easily adapted to the command-based paradigm by having Intake inherit from SubsystemBase.

只需让 Intake 继承 SubsystemBase，即可轻松将此示例改为 Command-Based 范式。

Our subsystem will be an Infinite Recharge intake mechanism containing a piston and a motor: the piston deploys/retracts the intake, and the motor will pull the Power Cells inside. We don't want the motor to run if the intake mechanism isn't deployed because it won't do anything.

我们的 subsystem 是一个 Infinite Recharge 赛季的 intake 机构，包含一个活塞和一台电机：活塞负责展开和收回 intake，电机负责将 Power Cells 吸入。intake 未展开时，不希望电机运行，因为那样不会产生效果。

To provide a “clean slate” for each test, we need to have a function to destroy the object and free all hardware allocations. In Java, this is done by implementing the `AutoCloseable` interface and its `.close()` method, destroying each member object by calling the member's `.close()` method - an object without a `.close()` method probably doesn't need to be closed.

为每次测试提供“全新状态”，需要有一个用于销毁对象并释放所有硬件分配资源的函数。在 Java 中，可通过实现 `AutoCloseable` 接口及其 `.close()` 方法来完成：对每个成员对象调用其 `.close()` 方法以销毁它；没有 `.close()` 方法的对象可能不需要关闭。

Note / 说明

Vendors might not support resource closing identically to the way shown here. See your vendor's documentation for more information as to what they support and how.

供应商对资源关闭操作的支持方式可能与这里的示例不同。有关支持范围和用法，请查阅供应商文档。

```
import edu.wpi.first.wpilibj.DoubleSolenoid;
import edu.wpi.first.wpilibj.PneumaticsModuleType;
import edu.wpi.first.wpilibj.examples.unittest.Constants.IntakeConstants;
import edu.wpi.first.wpilibj.motorcontrol.PWMSparkMax;
```

```

public class Intake implements AutoCloseable {
    private final PWMSparkMax m_motor;
    private final DoubleSolenoid m_piston;

    public Intake() {
        m_motor = new PWMSparkMax(IntakeConstants.kMotorPort);
        m_piston =
            new DoubleSolenoid(
                PneumaticsModuleType.CTREPCM,
                IntakeConstants.kPistonFwdChannel,
                IntakeConstants.kPistonRevChannel);
    }

    public void deploy() {
        m_piston.set(DoubleSolenoid.Value.kForward);
    }

    public void retract() {
        m_piston.set(DoubleSolenoid.Value.kReverse);
        m_motor.set(0); // turn off the motor
    }

    public void activate(double speed) {
        if (isDeployed()) {
            m_motor.set(speed);
        } else { // if piston isn't open, do nothing
            m_motor.set(0);
        }
    }

    public boolean isDeployed() {
        return m_piston.get() == DoubleSolenoid.Value.kForward;
    }

    @Override
    public void close() {
        m_piston.close();
        m_motor.close();
    }
}

```

Writing Tests / 编写测试

Important / 重要

Tests are placed inside the test source set: `/src/test/java/` for Java tests. Files outside that source root do not have access to the test framework - this will fail compilation due to unresolved references.

测试位于 test 源集中的 Java 测试目录 `/src/test/java/`。位于该源根目录之外的文件无法访问测试框架，因引用无法解析而编译失败。

In Java, each test class contains at least one test method marked with `@org.junit.jupiter.api.Test`, each method representing a test case. Additional methods for opening resources (such as our Intake object) before each test and closing them after are respectively marked with `@org.junit.jupiter.api.BeforeEach` and `@org.junit.jupiter.api.AfterEach`.

在 Java 中，每个测试类至少包含一个标有 `@org.junit.jupiter.api.Test` 的测试方法，每个方法代表一个测试用例。在各测试前打开资源（例如我们的 Intake 对象）和各测试后关闭资源的其他方法，分别标有 `@org.junit.jupiter.api.BeforeEach` 和 `@org.junit.jupiter.api.AfterEach`。

Each test method should contain at least one *assertion* (`assert*()` in Java). These assertions verify a condition at runtime and fail the test if the condition isn't met. If there is more than one assertion in a test method, the first failed assertion will crash the test - execution won't reach the later assertions.

每个测试方法应至少包含一个断言（Java 中为 `assert*()`）。这些断言会在运行时验证条件，并在条件不满足时使测试失败。如果一个测试方法包含多个断言，第一个失败的断言就会使测试中止，后续断言不会执行。

JUnit has multiple assertion types; the most common is equality: `assertEquals(expected, actual)`. When comparing numbers, a third parameter - `delta`, the acceptable error, can be given. In JUnit, these assertions are static methods and can be used without qualification by adding the static star import `import static org.junit.jupiter.api.Assertions.*`.

JUnit 提供多种断言类型，最常见的是相等性断言：`assertEquals(expected, actual)`。比较数值时，可以提供第三个参数 `delta`，表示可接受的误差。在 JUnit 中，这些断言是静态方法；添加静态星号导入 `import static org.junit.jupiter.api.Assertions.*` 后，即可不加类名直接使用。

Note / 说明

Comparison of floating-point values isn't accurate, so comparing them should be done with an acceptable error parameter (DELTA).

浮点值的比较并不精确，因此比较时应提供可接受的误差参数（DELTA）。

```
import static org.junit.jupiter.api.Assertions.assertEquals;

import edu.wpi.first.hal.HAL;
import edu.wpi.first.wpilibj.DoubleSolenoid;
import edu.wpi.first.wpilibj.PneumaticsModuleType;
import edu.wpi.first.wpilibj.examples.unittest.Constants.IntakeConstants;
import edu.wpi.first.wpilibj.simulation.DoubleSolenoidSim;
import edu.wpi.first.wpilibj.simulation.PWMSim;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

class IntakeTest {
    static final double DELTA = 1e-2; // acceptable deviation range
    Intake m_intake;
    PWMSim m_simMotor;
    DoubleSolenoidSim m_simPiston;

    @BeforeEach // this method will run before each test
    void setup() {
        assert HAL.initialize(500, 0); // initialize the HAL, crash if failed
        m_intake = new Intake(); // create our intake
        m_simMotor =
            new PWMSim(IntakeConstants.kMotorPort); // create our simulation PWM motor controller
        m_simPiston =
            new DoubleSolenoidSim(
                PneumaticsModuleType.CTREPCM,
```

```

        IntakeConstants.kPistonFwdChannel,
        IntakeConstants.kPistonRevChannel); // create our simulation solenoid
    }

    @SuppressWarnings("PMD.SignatureDeclareThrowsException")
    @AfterEach // this method will run after each test
    void shutdown() throws Exception {
        m_intake.close(); // destroy our intake object
    }

    @Test // marks this method as a test
    void doesntWorkWhenClosed() {
        m_intake.retract(); // close the intake
        m_intake.activate(0.5); // try to activate the motor
        assertEquals(
            0.0, m_simMotor.getSpeed(), DELTA); // make sure that the value set to the motor is 0
    }

    @Test
    void worksWhenOpen() {
        m_intake.deploy();
        m_intake.activate(0.5);
        assertEquals(0.5, m_simMotor.getSpeed(), DELTA);
    }

    @Test
    void retractTest() {
        m_intake.retract();
        assertEquals(DoubleSolenoid.Value.kReverse, m_simPiston.get());
    }

    @Test
    void deployTest() {
        m_intake.deploy();
        assertEquals(DoubleSolenoid.Value.kForward, m_simPiston.get());
    }
}

```

For more advanced usage of JUnit, see the framework docs.

有关 JUnit 的更高级用法，请参阅该框架的文档。

Running Tests / 运行测试

Note / 说明

Tests will always be run in simulation on your desktop. For prerequisites and more info, see [the simulation introduction](#).

测试始终在桌面计算机的仿真环境中运行。有关先决条件和更多信息，请参阅[仿真简介](#)。

For Java tests to run, make sure that your build.gradle file contains the following block:

要运行 Java 测试，请确保 build.gradle 文件包含以下代码块：

```
test {  
    useJUnitPlatform()  
    systemProperty 'junit.jupiter.extensions.autodetection.enabled', 'true'  
}
```

Use Test Robot Code from the Command Palette to run the tests. Results will be reported in the terminal output, each test will have a FAILED or PASSED/OK label next to the test name in the output. JUnit will generate a HTML document in `build/reports/tests/test/index.html` with a more detailed overview of the results; if there are any failed tests a link to render the document in your browser will be printed in the terminal output.

从命令面板选择 Test Robot Code 运行测试。结果会报告在终端输出中，每个测试的名称旁会有 FAILED 或 PASSED/OK 标签。JUnit 还会在 `build/reports/tests/test/index.html` 生成 HTML 文档，更详细地概述结果；如果有测试失败，终端输出中会打印可在浏览器中打开该文档的链接。

By default, Gradle runs the tests whenever robot code is built, including deploys. This will increase deploy time, and failing tests will cause the build and deploy to fail. To prevent this from happening, you can use Change Skip Tests On Deploy Setting from the Command Palette to configure whether to run tests when deploying.

默认情况下，每次构建机器人代码时 Gradle 都会运行测试，包括部署时。这会延长部署时间，而且测试失败会导致构建和部署失败。若不希望如此，可从命令面板使用 Change Skip Tests On Deploy Setting，配置部署时是否运行测试。